PROJECT DOCUMENTATION

Brain Tumor Detection with Artificial Intelligence

A plain-English + technical walkthrough of how the ML model was built, how it's wired into a real backend and website, exactly what every API call does, every library used and why, and a slide-by-slide explanation of the presentation deck.

> This document exists so you can actually explain this project in your own words — to a guide, a panel, or yourself at 2am before a viva — not just hand over files.

Parul University · AI & Data Science · April 2026

Contents

1. **The Big Picture** — what this project actually is, in one page
2. **How the ML Model Works** — dataset, transfer learning, two-stage training, Grad-CAM
3. **How the Model Gets Integrated Into the Website** — the files, the loading, the bug we found and fixed
4. **API Reference** — every endpoint, what it does, request/response examples
5. **Frontend Architecture** — how React turns an upload into a rendered result
6. **Every Technology & Library, and Why** — backend, frontend, and tooling
7. **Results, Explained** — what 97.10% actually means
8. **The PPT, Slide by Slide** — what's on each slide and why it's there
9. **Glossary** — every term you might get asked about

01 · ORIENTATION

The Big Picture

> **In one sentence:** this is a website where you upload a brain MRI image, and an AI model tells you whether it sees a tumour, what type, how confident it is, and *highlights on the image exactly which region made it think that.*

The project has three moving parts, and understanding how they connect is 90% of understanding the whole thing:

1. The Model (offline, one-time)

A neural network (VGG16, explained in Section 2) is trained on ~5,700 labelled MRI images until it can tell glioma / meningioma / pituitary / no-tumour apart. Training happens **once**, ahead of time, using `ml/train.py`. It produces saved model files on disk — training is not something that happens live on the website.

2. The Backend (Flask API)

`backend/app.py` is a small Python web server. It loads the saved model files **once when it starts up**, then sits there listening for HTTP requests. When it receives an image, it runs the model on it and sends back JSON with the answer.

3. The Frontend (React website)

The thing you actually see in a browser. It's a React app (in `frontend/`) with an upload box, a 3D animated hero section, and a results panel. It never touches the model directly — it just calls the backend's API over HTTP, the same way any website talks to any server.

Why split it this way?

This is the standard way real ML products are built: **training**, **serving**, and **UI** are three separate concerns. You can retrain the model without touching the website. You can redesign the website without retraining anything. You could even swap the frontend for a mobile app later and the backend wouldn't need to change at all.

The request, start to finish

When someone uploads a scan and clicks "Analyze," here is *literally everything* that happens:

1. The browser sends the image file to `POST /api/predict` on the Flask server.
2. Flask resizes the image to 224×224 pixels (the size VGG16 expects) and normalizes its pixel values the same way ImageNet-trained models expect.
3. The image is passed through the VGG16 convolutional layers, producing a compact "feature map" — a mathematical summary of the image's edges, textures and shapes.
4. That feature map goes through the classifier head (a small dense neural network) which outputs 4 probabilities that sum to 1 — one per tumour class.
5. **In the same pass**, Grad-CAM looks at which pixels in the feature map most influenced the winning class, and turns that into a heatmap image.
6. Flask packages the predicted class, the confidence, all 4 probabilities, and both images (original + heatmap overlay, as base64-encoded PNGs) into one JSON response.
7. React receives that JSON and renders it — the diagnosis banner, the toggle between original/heatmap, and the coloured probability bars.

This whole round-trip takes roughly 2-2.5 seconds on this machine's CPU (no dedicated GPU) — see Section 3 for why, and Section 7 for how we proved it stays fast even under repeated use.

02 · THE MODEL

How the ML Model Works

The dataset

We use the public Kaggle "Brain MRI" dataset, already sitting in `dataset/`, split into `Training/` and `Testing/` folders, one subfolder per class:

Class	Training images	Testing images
Glioma	1,321	300
Meningioma	1,339	306
No Tumour	1,595	405
Pituitary	1,457	300
Total	**5,712**	**1,311**

Why "transfer learning" instead of building a network from scratch?

5,712 training images is *tiny* by deep-learning standards — modern CNNs are normally trained on millions of images. Training a brand-new network from scratch on 5,712 images would overfit badly (memorize the training images instead of learning general patterns).

Transfer learning solves this by starting from **VGG16**, a CNN that Google/Oxford researchers already trained on ImageNet (1.4 million general photos — cats, cars, furniture, etc.). That training taught VGG16's early layers to recognise universal visual building blocks: edges, corners, textures, colour gradients. Those skills transfer to MRI scans too — an "edge" is an edge whether it's in a photo of a dog or a brain scan. We reuse those layers and only teach the network the *new*, tumour-specific part.

The two-stage training process

This is implemented in `ml/train.py` and is the most important engineering decision in the whole ML pipeline, so it's worth understanding in full:

Stage 1 — train only a small "head" on frozen features

- Load VGG16 with its ImageNet weights, **freeze** all of its convolutional layers (meaning: don't let training change them).
- Run every one of the 5,712+1,311 images through the frozen VGG16 **once**, and save the resulting feature maps ("bottleneck features") in memory.
- Train a small classifier head — `Flatten → Dense(256, relu) → Dropout(0.5) → Dense(4, softmax)` — directly on those cached features.
- Because the expensive part (running images through VGG16) only happens once instead of every epoch, this stage trains in **~11 minutes** instead of hours, even on a CPU.

Stage 2 — fine-tune the last block, end to end

- Unfreeze just the last convolutional block of VGG16 (`block5` , 3 conv layers) — the layers that encode the most task-specific, high-level features.
- Train the whole thing (unfrozen block5 + the classifier head) together, at a very low learning rate (`1e-5`), with live data augmentation (random flips, rotation, zoom) so the model sees a slightly different version of each image every epoch instead of memorizing exact pixels.
- This lets the network adapt its most specialised features specifically to MRI scans, recovering the last few points of accuracy Stage 1 couldn't reach — at the cost of this stage being much slower per epoch (full VGG16 forward + backward pass every time).

> **Why not just fine-tune everything from the start?** Two reasons: (1) it's dramatically slower with no CPU-friendly shortcut like bottleneck caching, and (2) with only ~5,700 images, unfreezing all 13 conv layers at once risks destroying the useful general-purpose features VGG16 already learned from ImageNet, before the model has learned anything tumour-specific to replace them with. Freezing most layers is a form of regularization.

What the model actually outputs

The final layer is `Dense(4, activation="softmax")` . Softmax turns 4 raw numbers into 4 probabilities that sum to exactly 1.0 — e.g. `[0.97, 0.01, 0.01, 0.01]` for glioma. The class with the highest probability is the prediction; that probability is reported as the "confidence."

Grad-CAM — how the heatmap is computed

Grad-CAM (Gradient-weighted Class Activation Mapping) answers: "*which pixels made the model choose this class?*" The idea:

1. Run the image forward through the network, but keep the feature map from the **last convolutional layer** (`block5_conv3`) instead of discarding it.
2. Compute the gradient of the winning class's score with respect to that feature map — this tells you how much each "channel" of the feature map influenced the decision.
3. Average those gradients per channel to get an importance weight, then compute a weighted sum of the feature map channels — this produces a small, coarse heatmap.
4. Resize that heatmap up to the original image size and blend it on top with a colour map (blue = low importance, red = high importance).

This is implemented once, reusably, in `ml/gradcam.py` , and is used both by the training script (to generate the example figure in the presentation) and by the live backend (Section 3).

Files produced by training

File	What it is
`model/vgg16_base.keras`	The trained VGG16 convolutional feature extractor
`model/classifier_head.keras`	The trained dense classifier (256 → 4)
`model/class_indices.json`	Ordered class name list, so index 0 always means "glioma," etc.
`model/evaluation_report.json`	Precision/recall/F1 per class on the held-out test set
`model/confusion_matrix.png` , `training_curves.png` , `gradcam_examples.png`	Report/presentation figures

03 · INTEGRATION

How the Model Gets Integrated Into the Website

> This is the part that usually confuses people: the model isn't "in" the website in the way a background image is in a website. It's a completely separate program (the Flask server) that the website talks to over the network — **even though, on your laptop, both happen to be running on the same machine.**

Why the model is split into two files, not one

You might expect one saved model file. We deliberately save **two** — `vgg16_base.keras` (the conv layers) and `classifier_head.keras` (the dense layers) — instead of one merged model. The reason is Grad-CAM: to build the heatmap, the code needs direct access to the *middle* of the network (the output of `block5_conv3`), not just the final answer. If the whole network were saved and loaded as a single nested Keras model, that intermediate layer becomes unreachable from outside — Keras treats a nested model as one opaque black box. Keeping the two halves separate means the backend can freely wire the conv layers' middle output into a small "Grad-CAM graph" alongside the head.

Loading the model — once, at server startup

Look at `backend/app.py`'s `_load_models()` function. The key line is at the very top:

```
def _load_models():
    if _state["base_model"] is not None:
        return  # already loaded — do nothing
```

This is a **lazy singleton pattern**: the very first request to hit the server triggers loading (or, in our case, we also call it once at startup so the server is ready immediately). Every request after that reuses the same already-loaded model objects sitting in memory — loading a 58 MB Keras model from disk takes real time, so doing it per-request would make every single prediction slow.

> **A real bug we found and fixed:** the first version of this code rebuilt the Grad-CAM "graph model" (the object connecting the middle layer to the output) *inside the prediction function, on every single request*, instead of once at startup. Individually that seemed cheap, but TensorFlow/Keras keeps internal bookkeeping every time you wire layers into a new graph, and doing it repeatedly in a long-running server caused each request to get progressively slower — 2.6s, then climbing past 8.8s and rising the longer the server had been running, discovered when we load-tested the live API against all 1,311 test images. Left alone, a real user session would have ground the site to a halt.

The fix: build the Grad-CAM graph model exactly once, right after the model files load, and store it in the server's shared `_state` dictionary alongside the base model and head model. Every request now reuses the same cached object. We verified the fix by hammering the live endpoint 25+ times in a row and confirming latency stays flat at ~2.4s instead of climbing.

What gets cached in `_state`

Key	What it holds
`base_model`	Loaded VGG16 conv layers
`head_model`	Loaded classifier head
`grad_model`	The Grad-CAM graph model, built once
`class_names`	["glioma","meningioma","notumor","pituitary"]
`preprocess_input`	VGG16's exact ImageNet pixel-normalization function
`load_error`	Set if the model files are missing/broken, so every endpoint can report it cleanly instead of crashing

Warm-up: exercising the real code path before any user hits it

Right after loading, the server runs `compute_gradcam()` once on a dummy all-zero image. This isn't a no-op — TensorFlow does some one-time "tracing" work the first time a particular computation graph runs. Doing that once during startup means any first-call slowness (or first-call bugs) get caught while the server is starting up, not on a real user's first upload.

04 · API REFERENCE

Every API Endpoint, Explained

All three endpoints live in `backend/app.py` and are served at `http://localhost:5000`. The React dev server proxies any request to `/api/*` straight through to this Flask server (configured in `frontend/vite.config.js`) — that's why the frontend code just calls `fetch("/api/predict")` with no hostname, and it works without CORS headaches in development. `flask-cors` is also enabled server-side as a second safety net for any other origin.

GET /api/health

Purpose: a lightweight liveness/readiness check. The frontend calls this once when the page loads to show "Backend: connected, model loaded" in the footer.

What it does: calls `_load_models()` (so hitting `/api/health` is actually what triggers the very first model load if the server just started) and reports the result.

Example response (model ready):

```
{
  "status": "ok",
  "model_loaded": true,
  "error": null
}
```

Example response (before training finished):

```
{
  "status": "ok",
  "model_loaded": false,
  "error": "Trained model files not found in model/. Run ml/train.py first."
}
```

GET /api/model-info

Purpose: gives the frontend the class list and the model's own offline evaluation metrics, so the UI can show "Test-set accuracy: 97.10%" without hard-coding that number in the React code.

What it does: reads `model/evaluation_report.json` (produced once by `ml/train.py`) off disk and returns it alongside the class names. Returns HTTP 503 if the model isn't loaded yet.

Example response (trimmed):

```
{
  "classes": ["glioma", "meningioma", "notumor", "pituitary"],
  "metrics": { "accuracy": 0.9710, "glioma": {"precision": 0.993, ...}, ... }
}
```

POST /api/predict — the important one

Purpose: the entire product. Takes an uploaded MRI image, returns a diagnosis, confidence, per-class probabilities, and a Grad-CAM heatmap.

Request: `multipart/form-data` with one field named `image` (a PNG/JPG/BMP file, max 16 MB). This is exactly what a browser's `<input type="file">` + `FormData` produces, and what `frontend/src/api.js`'s `predictImage()` sends.

Step by step, what the handler does:

1. Validates a file was actually attached and has an allowed extension → else HTTP 400.
2. Opens it with Pillow, converts to RGB (handles grayscale/RGBA scans uniformly).
3. Makes a *display* copy, capped at 512px on the longest side (for a reasonably sized image to send back — no point shipping a multi-megapixel PNG to the browser).
4. Makes a separate *model* copy, resized to exactly 224×224 and run through VGG16's `preprocess_input` (converts RGB→BGR and subtracts the ImageNet per-channel mean — the exact preprocessing VGG16 was originally trained with; skipping this would silently hurt accuracy).
5. Calls `compute_gradcam()` **once** — this single call returns the heatmap, the predicted class index, *and* the full probability vector, because Grad-CAM's own forward pass through the cached `grad_model` already computes the classifier's output as a side effect (no need for a second, separate prediction pass).
6. Blends the heatmap onto the *display*-resolution image using OpenCV's JET colour map (via `overlay_heatmap_on_image()`).
7. Encodes both the original and the heatmap-overlay images as base64 PNG data URIs — this lets the JSON response carry the actual images inline, so the browser can put them straight into an `<img src="…">` tag with zero extra requests.

Example response (trimmed — the two image fields are normally tens of KB of base64 each):

```
{
  "prediction": "glioma",
  "tumor_detected": true,
  "confidence": 0.998,
  "probabilities": {
    "glioma": 0.998, "meningioma": 0.001,
    "notumor": 0.000, "pituitary": 0.001
  },
  "original_image": " ... ",
  "heatmap_overlay": " ... "
}
```

Error responses: HTTP 400 for a missing/invalid file, HTTP 503 if the model isn't loaded, with a plain `{"error": " ... "}` body in both cases — the frontend shows this text directly under the Analyze button.

05 · FRONTEND

Frontend Architecture

The React app lives in `frontend/src/`. Nothing here does any ML — its entire job is: collect a file, call the API, and render whatever JSON comes back.

Component map

File	Responsibility
`App.jsx`	Top-level state: the analysis history list, the currently selected result, loading/error flags. Renders Hero → workspace (Upload + History + Result) → footer.
`components/Hero.jsx`	Nav bar, headline, live stats pulled from `/api/model-info`, and the 3D canvas.
`components/BrainScene.jsx`	The Three.js/React-Three-Fiber scene — a distorted icosahedron core, a wireframe shell, and a rotating particle field. Purely decorative, no data dependency.
`components/UploadPanel.jsx`	Drag-and-drop + file picker, image preview, the "Analyze Scan" button that triggers the API call.
`components/ResultPanel.jsx`	Renders the diagnosis banner, the Original/Grad-CAM toggle, and the probability bars for whichever result is currently selected.
`components/ProbabilityBars.jsx`	The 4 coloured bars, one per class, each with a fixed colour (see Section 6) so a class always looks the same everywhere.
`components/HistoryPanel.jsx`	List of everything analyzed this session; clicking an entry re-displays that result.
`api.js`	The only file that knows the API's URLs — `checkHealth()`, `getModelInfo()`, `predictImage(file)`.

State flow for one upload

1. User drops/selects a file → `UploadPanel` stores it locally and shows a preview (this never touches the network — `URL.createObjectURL()` is a purely client-side preview).
2. User clicks Analyze → `App.handleAnalyze()` sets `loading=true`, calls `predictImage(file)`.
3. `api.js` wraps the file in a `FormData` and does `fetch("/api/predict", {method: "POST", body: formData})`.
4. On success, the JSON result is pushed onto the `history` array and marked as the active result — React re-renders `ResultPanel` with it.
5. On failure, the error message is stored and shown under the button; `loading` resets either way.

Everything is plain React state (`useState`) — no Redux, no external state library. The app is small enough that it doesn't need one.

06 · TECH STACK

Every Technology & Library, and Why

Backend / Machine Learning

Library	What it's for in this project
Python 3.13	The language everything backend/ML is written in.
TensorFlow / Keras 3	Defines, trains, saves, loads and runs the VGG16 model and classifier head. Also provides the `image_dataset_from_directory` data pipeline and the `GradientTape` API Grad-CAM is built on.
NumPy	All the array math glue — reshaping images into batches, computing softmax argmax, normalizing pixel arrays.
Pillow (PIL)	Opens uploaded images in any format, converts to RGB, resizes for the model vs. for display.
OpenCV (opencv-python-headless)	Resizes the Grad-CAM heatmap to the display image's size, applies the JET colour map, and blends it into the overlay.
scikit-learn	Computes precision/recall/F1 (`classification_report`) and the confusion matrix during evaluation — not used at inference time.
Matplotlib	Draws the confusion matrix, training curves, and Grad-CAM example grid saved into `model/` .
Flask	The web server framework — turns Python functions into HTTP endpoints (`/api/health` , `/api/model-info` , `/api/predict`).
Flask-CORS	Adds the headers that let a browser on one origin (the Vite dev server) call an API on another origin (Flask) without being blocked by the browser's same-origin policy.
Requests	Used only in the testing script (`ml/evaluate_api.py`) to drive the live API like a real client, for the full-dataset integration test.

Frontend

Library	What it's for in this project
React 19	Component-based UI — every visible piece (upload box, result card, probability bars) is a component that re-renders when its data changes.
Vite	The dev server and build tool. Also configured to proxy `/api/*` to the Flask backend during development.
Three.js	The underlying 3D graphics engine (WebGL) powering the hero visualization.
@react-three/fiber	Lets Three.js scenes be written as React components instead of imperative JS — the `<Canvas>`, meshes and lights in `BrainScene.jsx`.
@react-three/drei	Ready-made 3D helpers — `MeshDistortMaterial` (the organic wobble on the core shape), `Points` / `PointMaterial` (the particle field), `Float` (gentle bobbing animation).
Framer Motion	Entrance animations for the hero text and the fade/slide transition when a new result appears.

Tooling used to build & verify the project

Tool	What it's for
Playwright	Headless-browser automation used to actually click through the live website and confirm uploads/results render correctly — real end-to-end testing, not just unit tests.
python-pptx	Generates the 15-slide presentation programmatically from real project data (no manual slide design).

07 · RESULTS

Results, Explained in Plain Terms

> **Overall test accuracy: 97.10%** — out of 1,311 MRI scans the model had never seen during training, it got the tumour type right 1,273 times.

Precision, recall and F1 — what they actually mean here

- **Precision** — of every scan the model *called* "glioma," what fraction really were glioma? Low precision means false alarms.
- **Recall** — of every scan that *actually was* glioma, what fraction did the model catch? Low recall means missed cases — the scarier failure mode in medicine.
- **F1** — the balance of the two, useful when you care about both equally.

Class	Precision	Recall	F1	What this tells us
Glioma	99.3%	92.7%	95.9%	Very few false alarms, but ~7% of real gliomas were missed — the model's weakest recall.
Meningioma	92.4%	95.8%	94.1%	Catches almost all real meningiomas, at the cost of a few false positives — likely gliomas being confused for meningiomas and vice versa, since they can look visually similar (also flagged in the written report, Ch. 6.4).
No Tumour	99.5%	100%	99.8%	Essentially perfect — healthy scans look distinctly different from any tumour class.
Pituitary	96.7%	99.0%	97.9%	Strong on both counts.

Why we didn't just trust the training script's own number

97.10% comes from `ml/train.py` evaluating the model directly, in-process, in Python. That proves the *model* is accurate — it does not prove the *website* is wired up correctly. A subtly wrong preprocessing step, a class-index mismatch, or a bug in the API's image handling could all silently produce wrong answers through the website even with a perfectly good model underneath. So we separately wrote `ml/evaluate_api.py`, which sends every one of the 1,311 test images through `POST /api/predict` exactly like the real frontend does — over real HTTP, through the real Flask code path — and independently recomputes accuracy from the live responses. This is what caught the Grad-CAM performance bug described in Section 3.

08 · THE DECK

The PPT, Slide by Slide

What's on each of the 15 slides in `Brain_Tumor_Detection_Presentation.pptx`, and what you'd say while presenting it.

01 — Title Slide

Shows: Project name, team, guide, university, date.

Say: One-line framing — "an AI system that classifies brain tumours from MRI and explains its own reasoning."

02 — What is a Brain Tumor?

Shows: Benign vs malignant, why MRI is the standard imaging tool, the 3 major tumour types.

Say: This sets up the medical context before touching any tech.

03 — The Problem We're Solving

Shows: Radiologist workload, inter-observer variability, why delayed diagnosis matters.

Say: This is the "why does this project need to exist" slide — AI as a second opinion, not a replacement.

04 — Objectives & Scope

Shows: 5 concrete objectives, plus an explicit in-scope/out-of-scope split.

Say: Be explicit this is a screening aid, not a diagnostic or treatment tool — this line matters if a panel asks about clinical claims.

05 — Literature Survey

Shows: 6 prior approaches (F-Transform, autoencoders, ResNet18, MobileNet, VGG16+RCNN, Gaussian CNN) condensed to one line each.

Say: These informed the choice of VGG16 transfer learning as a proven, accuracy/speed-balanced approach for this exact problem.

06 Dataset

Shows: The train/test split table, and one real sample image per class.

Say: Public Kaggle dataset, already class-balanced-ish and pre-split 70/30-ish per class.

07 Proposed Methodology

Shows: The 5-step pipeline diagram (Input → Preprocess → Augment → CNN Features → Classify), plus the convolution formula and thresholding rule.

Say: This is the high-level version of Section 2 of this document.

08 Model Architecture

Shows: The VGG16 layer diagram (frozen blocks 1-4 → fine-tuned block 5 → dense head) and the training config table (optimizer, loss, regularization).

Say: This is where you explain the two-stage training decision from Section 2 if asked "why not just train it normally?"

09 System Architecture

Shows: The 4-tier diagram — React Client → Flask REST API → VGG16+Grad-CAM → Model Store — and the request-flow bullets.

Say: This is Section 3 and 4 of this document, visualised. Good slide to point at if asked "how does the website actually use the model?"

10 Implementation & Tech Stack

Shows: Backend and frontend library lists side by side.

Say: The short version of Section 6 of this document.

11 Grad-CAM: Explainable AI

Shows: The problem/fix/payoff of explainability, and a real 2×4 grid of Grad-CAM outputs — one correctly-classified example per class, heatmap under original.

Say: This is usually the most memorable slide — point out the heatmap actually sits on the tumour location, not a random spot.

12 — Results — 97.10% Test Accuracy

Shows: The per-class precision/recall/F1 table and the real confusion matrix image.

Say: Use Section 7 of this document to explain precision vs. recall if asked, and mention the glioma/meningioma confusion pattern visible in the matrix.

13 — Live Demo — The Working Website

Shows: Real screenshots — the 3D hero section, and an actual analysis result (pituitary, 100% confidence, heatmap overlay, probability bars).

Say: Ideally, don't just show this slide — actually open the website live if you can. This slide is the fallback.

14 — Conclusion & Future Work

Shows: What was built (left) vs. where it could go next — segmentation, multi-institution data, DICOM/PACS integration, tumour grading (right).

Say: Be upfront about the current limitations (single public dataset, binary-ish classification, no clinical validation) — panels respond well to self-aware scoping.

15 — Thank You

Shows: Closing slide, team credit, 3 key references from the literature survey.

Say: Invite questions — you now have this whole document behind you to answer almost anything that comes up.

09 · REFERENCE

Glossary

CNN (Convolutional Neural Network)

A neural network architecture specialised for images — it learns small filters that slide across the image detecting patterns like edges and textures, then stacks layers of those to build up more complex shapes.

Transfer learning

Reusing a model already trained on one large task (ImageNet) as a starting point for a different, smaller task (tumour classification), instead of training from random weights.

Fine-tuning

Continuing to train some or all of a pre-trained model's layers on new data, usually at a low learning rate, to specialise it further.

Bottleneck features

The output of a frozen network's convolutional layers, cached once so the (expensive) conv computation doesn't need to be repeated every training epoch.

Epoch

One full pass through the entire training dataset during training.

Softmax

A function that converts raw output numbers into probabilities that sum to 1 — used for the final 4-class prediction.

Confidence

The softmax probability of the predicted (winning) class — how "sure" the model is.

Grad-CAM

Gradient-weighted Class Activation Mapping — a technique for visualising which parts of an input image most influenced a CNN's prediction, by tracing gradients back to the last convolutional layer.

Precision / Recall / F1

Standard classification metrics — see Section 7 for the plain-English explanation used in this project.

Confusion matrix

A table showing, for every true class, how many scans were predicted as each possible class — the diagonal is correct predictions, everything off it is a mistake.

REST API

A convention for structuring web server endpoints (URLs + HTTP methods) so a client (like our React app) can request data or trigger actions in a predictable way.

CORS

Cross-Origin Resource Sharing — the browser security rule that blocks a webpage from one address from freely calling an API on a different address, unless the server explicitly allows it.

Multipart/form-data

The HTTP request format used for uploading files (as opposed to plain JSON), because it can carry binary file data alongside form fields.

Base64 data URI

A way of embedding binary data (like an image) directly as text inside a JSON response or an HTML `src` attribute, avoiding a separate file request.

Generated as project documentation for the Brain Tumor Detection with AI final year project — Parul University, AI & Data Science, April 2026.